

2025 오픈소스프로그래밍 프로젝트 최종보고서

프로젝트명	걸음걸이 데이터를 통한 치매 고위험군 예측 모델 연구			
팀명	옥수수수염차 (11조)	팀장	학번	이름
			2314995	이다연
수행기간	2025. 05. 15 ~ 2025. 06. 22 (학기말까지)	팀원	2313498	이세은
			2315398	정연재
			2310584	허여령

가. 목표 및 기대효과

- 기술개발 배경
 - 고령화로 인해 치매 유병률이 증가하고 있으며, 특히 65세 이상 인구의 약 10%가 치매 초기 증상을 보이고 있다.
 - 파킨슨병은 운동 기능 저하를 초기 증상으로 보이며, 이후 인지 기능 저하로 진행될 수 있어 특정 유형의 치매와 관련이 깊다.
 - 병원 방문이 어려운 고령층을 위해, 일상에서 얻을 수 있는 걸음걸이 데이터를 활용한 치매 고위험군 선별 시스템 개발이 필요하다.
 - 프로젝트 초기에 알츠하이머 예측을 목표로 했으나, 데이터 특성상 파킨슨병 기반의 위험도 분석이 더 타당하다고 판단되어 프로젝트 명을 Dementia Diagnosis에서 "걸음걸이 데이터를 통한 파킨슨병 기반 치매 고위험군 예측 모델 연구"로 변경하였다.
- 기술개발 목표
 - 웨어러블 기기 기반의 걸음걸이 데이터를 활용하여, 딥러닝 분류 모델을 통해 치매 고위험군을 4단계 위험도로 정량 분류하고, 실제 개인 데이터에 적용 가능한 분류 시스템 프로토타입을 개발한다.
- 기대효과
 - 웨어러블 기기를 통해 수집된 걸음걸이 데이터로 치매 위험도를 단계별로 분류하는 정량적 모델링 구현
 - 운동 기능 기반 생체 신호 분석을 통해 향후 치매뿐만 아니라 다양한 신경계 질환 조기 진단 기술로 확장 가능
 - 병원 방문이 어려운 고령층, 의료 인프라가 부족한 지역에서도 운동 기반의 치매 조기 예측이 가능해진다.

나. 프로젝트 개발 내용

1. 연구개발 내용

본 연구개발에서는 [그림 1]과 같이 걸음걸이 데이터를 기반으로 치매 고위험군을 예측하는 딥러닝 모델을 개발하고자 하며, 이를 위해 다음 두 모듈을 주요 설계 기준으로 정한다.

- Gait Data Preprocessing 모듈 (걸음걸이 데이터 전처리 모듈)
- Gait Dementia Risk Prediction 모듈 (치매 고위험군 예측 모듈)

2. Gait Data Preprocessing 모듈 (걸음걸이 데이터 전처리 및 특징 추정 모듈)

(1) 걸음걸이 데이터 구조 및 특성 분석

본 프로젝트에서는 CSV파일 형태로 수집된 보행자 움직임 데이터를 사용한다. 이 데이터는 웨어러블 기기를 통해 수집되었으며, 활동 관련 데이터, 활동 지속 시간 데이터 등 다양한 수치 데이터가 포함되어 있다.

모델 학습을 위해 전체 변수 중 다음 유의미한 8개 칼럼을 입력 feature로 선정했다.
'activity_score', 'activity_steps', 'activity_inactive', 'activity_high', 'activity_score_meet_daily_targets',
'activity_score_move_every_hour', 'activity_daily_movement', 'activity_total'

(2) 데이터 전처리 및 정규화 처리

- 결측치 및 이상치 확인

수집된 데이터에 결측치는 없었고, 이상치를 확인하였으나, 각 feature별 범주가 다양한 관계로, K-means 클러스터링으로 이상치 제거 대신 보완적 처리로 진행된다.

- 전처리 파이프라인 설계 (Z-score 기반 정규화)

StandardScaler를 이용하여 모든 수치형 feature에 대해 평균 0, 표준편차 1로 정규화함으로써 학습을 안정화시키고 비교가 가능하도록 만들었다..

- Silhouette Score을 통한 클러스터 수 검증

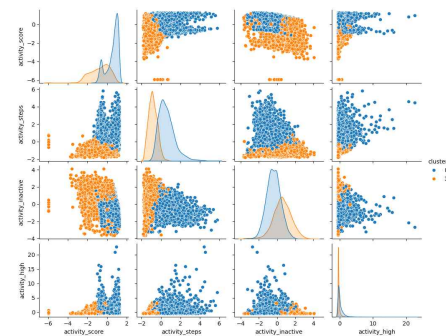
KMeans의 클러스터 수 k를 2~10까지 변화시키며 Silhouette Score를 계산해 군집 품질을 수치적으로 평가함. 이를 통해 최적의 k=2 값을 선택함.

- 통계적 특징 추출 (KMeans 클러스터링)

KMeans 클러스터링을 적용하여 유사한 걸음걸이 패턴을 갖는 그룹을 나눔. 분류 0번은 정상 활동, 1번은 신체 기능 저하 의심군으로 분류되었다.

Cluster 0 : 정상적인 활동 수준을 보이는 그룹 (저위험군)

Cluster 1 : 활동량이 낮고, 비활동 시간이 긴 신체기능 저하 (치매 고위험 의심군)



[그림 1. K-means 클러스터링 결과]

- XGBoost 기반 지도학습용 라벨 생성

클러스터 결과를 타깃 라벨(y), 활동 지표를 입력값(x)로 하여 XGBoost 분류 모델을 학습했다. 이후 SHAP 값을 통해 각 feature가 위험군 분류에 얼마나 기여했는지를 계산한다.

- SHAP 기반 위험도 점수 생성 및 등급화

각 샘플에 대해 SHAP의 절댓값 총합을 계산하여 위험도 기여도 점수를 산출한다.

위험도 점수를 0~100%로 정규화한 후, 다음과 같은 기준으로 4단계 위험도 등급(risk_level)으로 분류했다.

위험도 등급	범위(%)	설명
0단계(level 0)	0~40	치매 저위험군
1단계(level 1)	41~70	중하 위험군
2단계(level 2)	71~80	중상 위험군
3단계(level 3)	81~100	치매 고위험군

- PyTorch 학습용 TensorDataset 및 Dataloader 구성

최종적으로 risk_level을 레이블로 지정하여 학습/검증/테스트셋 분할하였다. NumPy 배열을 torch tensor로 변환 후 TensorDataset을 구성하였다.

모델 학습을 위한 DataLoader로 래핑하여 딥러닝 학습 준비가 완료되었다.

3. Gait Dementia Risk Prediction 모듈 (치매 고위험군 예측 모듈)

(1) 딥러닝 모델 설계 분석

본 프로젝트에서는 웨어러블 기기로부터 얻은 활동 데이터를 기반으로, 고위험군의 신체 기능 저하 정도를 0~3단계 위험도로 분류하고자 하였다. 이를 위해 PyTorch 기반의 MLP 모델을 설계하여 예측 성능을 검증하였다.

(2) 초기모델 설계 및 학습 세팅

- 모델 구조

[그림 2]의 모델에서 입력층에서는 총 8개의 feature를 입력받도록 설정, 은닉층은 64개의 노드에서 시작해 32개로 점진적으로 축소되는 구조로 구성되었다. 각 은닉층에는 ReLU 활성화 함수를 적용하여 비선형성을 확보하였다. 출력층은 총 4개의 노드로 구성하여 각 클래스(0~3)에 해당하는 위험수준 확률을 산출한다. 초기 모델 구조는 과적합을 방지하고 모델의 일반적인 경향을 확인하기 위해 간단하게 설계됐으며, 이에 Dropout은 과적합 여부를 확인 후 도입 예

```
self.net = nn.Sequential(  
    nn.Linear(8, 64),  
    nn.ReLU(),  
    nn.Linear(64, 32),  
    nn.ReLU(),  
    nn.Linear(32, 4),  
)
```

[그림 2. 초기모델 구조]

- 하이퍼파라미터

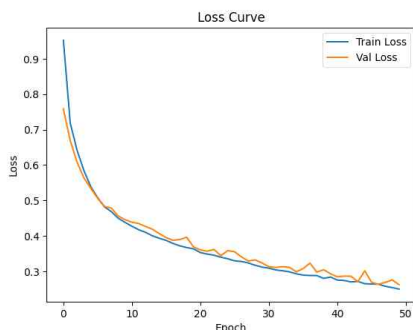
학습률은 0.001로 설정하였으며, 이는 Adam 옵티마이저의 권장 기본값으로 안정적인 수렴을 기대할 수 있다. Batch size는 64로 설정하여 계산 효율성과 일반화 성능 간의 균형을 고려하였고, Epoch 수는 50으로 초기 학습 성능을 탐색하고 향후 개선 방향을 파악하기 위한 예비적 설정이다.

- 학습 구조

각 epoch 마다 train loss와 validation loss를 추적하여 과적합 여부 실시간 확인하였다. model.eval() 및 torch.no_grad()를 사용하여 평가 시 gradient 계산을 방지했다.

- 성능 평가

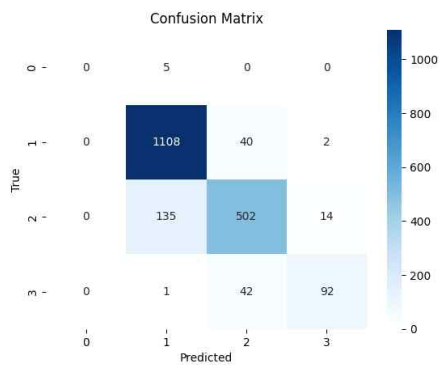
성능 평가는 테스트셋에서 모델이 예측한 결과를 바탕으로 classification_report()와 confusion_matrix()를 통해 수행하였다. 각 클래스별 precision, recall, f1-score 등의 주요 지표를 확인하고, confusion matrix를 시각화함으로써 클래스 간의 혼동 양상도 함께 분석했다.



loss curve 분석

-train loss와 validation loss가 동일한 추세로 안정적 감소, 과적합 없고 학습이 잘 수렴됨.

[그림 3. 초기 loss curve 그래프]



Classification Report

	precision	recall	f1-score	support
0	0.0000	0.0000	0.0000	5
1	0.8871	0.9635	0.9237	1150
2	0.8596	0.7711	0.8130	651
3	0.8519	0.6815	0.7572	135
accuracy			0.8769	1941
macro avg	0.6496	0.6040	0.6235	1941
weighted avg	0.8731	0.8769	0.8726	1941

[그림 4. 초기 confusion matrix]

confusion matrix 분석

- class0 : 5건 전부 오분류
- class1 : 아주 정확히 잘 맞춤(1108/1150)
- class2 : 절반 이상 맞췄지만 class1이랑 혼동하는 경우 많음
- class3 : class2와 혼동 많음 일부 1과도 혼동

초기 모델의 첫 번째 성능 결과에서는 class1에 한하여, 양호한 성능을 보였다. 특히 loss curve 분석 결과, train loss와 validation loss가 유사한 추세로 안정적으로 감소하는 양상을 나타냈다. 이는 과적합 없이 모델이 학습 데이터와 검증 데이터 모두에서 잘 수렴하고 있음을 의미한다.

(3) 초기 모델링 문제 원인 분석과 해결

Class 0(치매 저위험군)의 경우, 전체 데이터 내 샘플 수가 매우 적고, 사용된 feature들의 분포가 다른 클래스들과 유사하여 모델이 해당 클래스를 구분할 수 있는 명확한 분류 근거를 찾기 어려웠다. 이로 인해 예측 정확도가 크게 떨어졌으며, 대부분 다른 클래스로 오분류 되는 현상이 발생하였다.

또한 class 2와 class 3 사이에서는 혼동이 자주 나타났는데, 이는 실제 데이터에서도 두 위험 단계 간의 경계가 뚜렷하게 구분되지 않고 점진적으로 형성되기 때문이다. 이러한 특성상 feature 기반으로 명확한 컷오프(cut-off)를 설정하기 어려워 모델이 경계선상의 데이터를 올바르게 분류하는데 어려움을 겪는 것으로 분석된다.

이를 해결하기 위해 아래의 방법을 사용하였다.

- drop 추가 등의 모델구조 복잡도 상승, 에폭, 배치 수정
- drop 추가 등의 모델구조 복잡도 상승

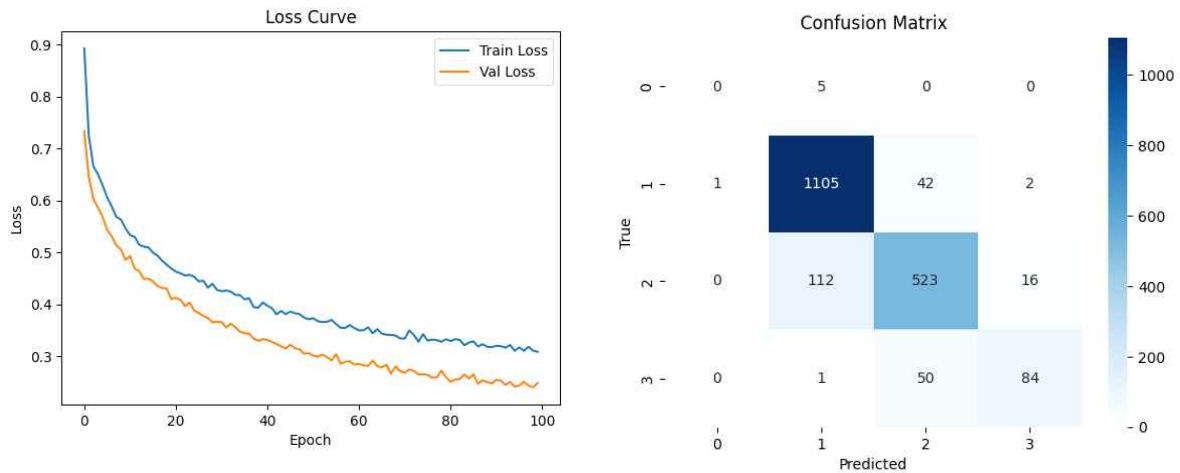
```
self.net = nn.Sequential(
    nn.Linear(8, 256),
    nn.ReLU(),
    nn.Dropout(0.4),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 4),
)
```

모델 구조를 보다 깊게 설계하여 은닉층을 추가하고, Dropout 기법을 도입하여 학습을 진행하였다. 은닉층의 확장은 모델의 표현력을 높여 복잡한 패턴을 더 효과적으로 학습할 수 있도록 하였으며, Dropout은 학습 과정에서 불필요한 과적합을 방지하고 모델의 일반화 성능을 향상시키는데 기여하였다. 이를 통해 클래스 간 구분 성능을 개선하고, 특히 경계가 모호한 클래스에 대한 분류 정확도를 높이하고자 하였다.

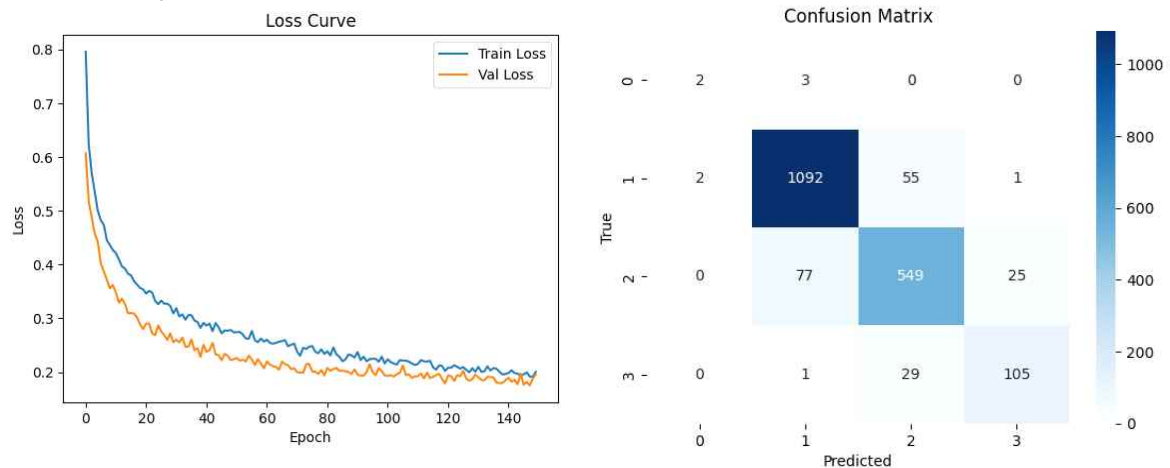
[그림 5. 최종 모델 구조]

- 에폭, 배치 수정

[그림 6]은 drop 1개 추가, 에폭 100, 배치 32일 때이고 [그림 7]는 drop 2개 추가, 에폭 150, 배치 32일 때이다. 각각 에폭 50과 100에서 시행했을 때 과적합이 발생하지 않았으나 150으로 실행 했을 때 과적합이 발생하였다. 이 두 결과를 비교해 보니 최종 에폭 120정도가 적합하는 판단을 내렸다.



[그림 6. drop 1개 추가, 에폭 100, 배치 32에서의 loss curve, confusion matrix]



[그림 7. drop 2개 추가, 에폭 150, 배치 32에서의 loss curve, confusion matrix]

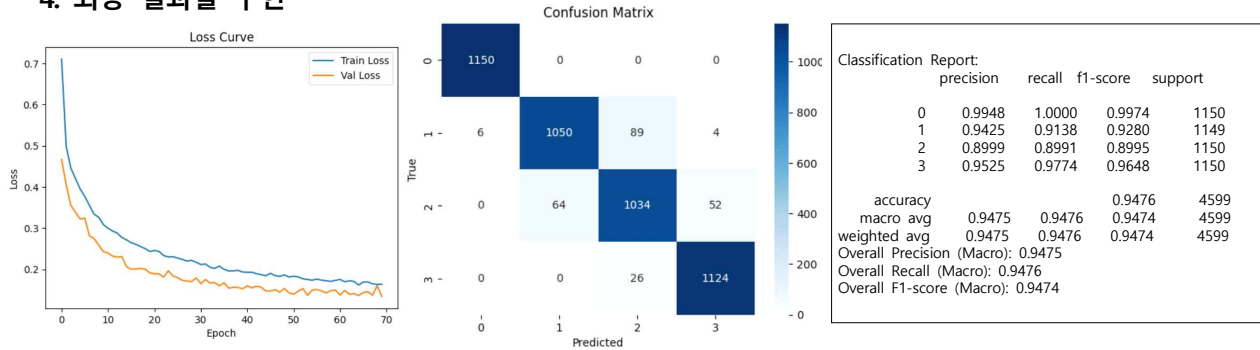
• SMOTE

Class 0(치매 저위험군)은 전체 데이터에서 차지하는 비중이 작아 분류에 어려움이 있었다. 이를 해결하기 위해 SMOTE를 적용하였다. SMOTE는 기존 샘플을 기반으로 소수 클래스 데이터를 인위적으로 생성해 클래스 간 불균형을 해소하고, 모델이 class 0에 대한 학습 기회를 충분히 확보할 수 있도록 도와 예측 성능 향상에 기여하였다.

• 클래스 가중치 계산

소수 클래스에 대한 분류 성능을 향상시키기 위해, 각 클래스의 샘플 수에 반비례하여 클래스별 가중치를 계산하고 이를 PyTorch의 손실 함수에 적용하였다. 학습 중 소수 클래스의 오류에 더 큰 패널티를 부여함으로써 클래스 간 학습 균형을 맞추고, 기존에 성능이 낮았던 class 0에 대해 모델이 보다 민감하게 학습할 수 있도록 설계되었다.

4. 최종 결과물 구현



[그림 8. 최종 모델의 loss curve, confusion matrix, classification Report]

학습 과정에서의 loss curve를 보면, train loss와 validation loss가 모두 안정적으로 감소하고 있으며, 후반부에는 두 손실 값이 거의 수렴하는 양상을 보인다. 이는 학습이 잘 진행되었고, 과적합 없이 모델이 일반화 성능을 잘 유지하고 있음을 의미한다.

Confusion matrix 분석 결과, 네 개의 클래스 모두에서 비교적 균형 잡힌 분류 성능을 보여준다. 특히 class 0과 class 3은 매우 정확하게 분류되었으며, class 1과 class 2 사이에서는 다소 혼동이 있으나 큰 성능 저하는 없는 수준이다.

Classification Report에 따르면, 전체 정확도는 94.76%에 달하며, macro 평균 precision, recall, f1-score 모두 0.947 수준으로 매우 우수한 성능을 나타냈다. 특히 소수 클래스였던 class 0에서도 precision 0.9948, recall 1.0000, f1-score 0.9974로 좋은 예측 결과를 보였다.

다. 추진일정 및 업무분장

1) 역할 분담 계획

구 분	성 명	담당 업무	세부개발 사항
팀장	이다연	데이터 라벨링, 모델 성능 확인	(1)Shillouette score (2)K-means clustering (3) 모델 성능 확인용 function check 최종 파이썬 파일 제작
팀원1	이세은	데이터 전처리, 모델링	(1)결측치, 이상치 탐색 (2)Normalized (3) 딥러닝 모델 성능 향상 개발(smote, 클래스 가중치 계산 모델 복잡도 향상등)
팀원2	정연재	데이터 라벨링, 모델 성능 확인	(1)라벨링, 데이터셋 분리 (2) Shap 보조 (3) pth파일 준비 (4) 모델 성능 확인용 Sample Data 제작
팀원3	허여령	데이터 전처리, 모델링	(1)데이터 정리 (2)SHAP (3)Clustered Data 파일 (4) MLP 초기 모델 구현 (5) epoch,배치 실험 (기타 : 전체 과정 가이드 및 보조)

2) 일정 계획

세부 개발내용	과제 수행 기간					
	5월 3주	5월 4주	5월 5주	6월 1주	6월 2주	6월 3주
1. 데이터 준비 및 전처리						
2. 데이터 라벨링						
3. 딥러닝 모델링						
4. 모델 성능확인						

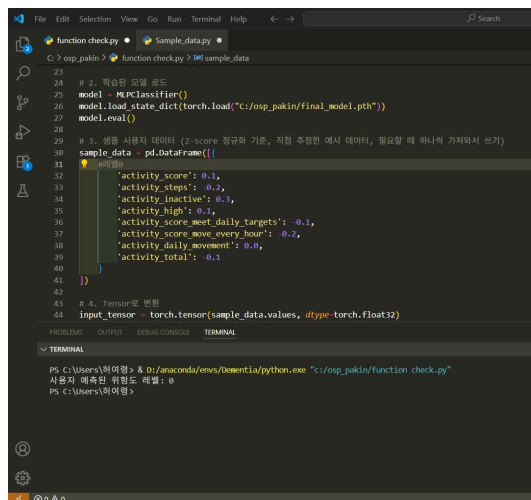
다. 프로젝트 개발 결과

- 최종결과물 내용
 - 결과물 예시

최종적으로 본 프로젝트에서는 학습된 딥러닝 모델(final_model.pth)을 활용하여, 사용자가 임의로 입력한 걸음걸이 데이터를 기반으로 치매 위험도를 class 0에서 class 3단계까지 분류해주는 기능을 구현하였다. class 0에서 class 3를 0단계에서 3단계로 명명한 뒤, VSCode 환경에서 파이썬 파일(function check.py)을 실행, 터미널에서 확인하는 방식으로 간단한 테스트 환경을 구성했다.

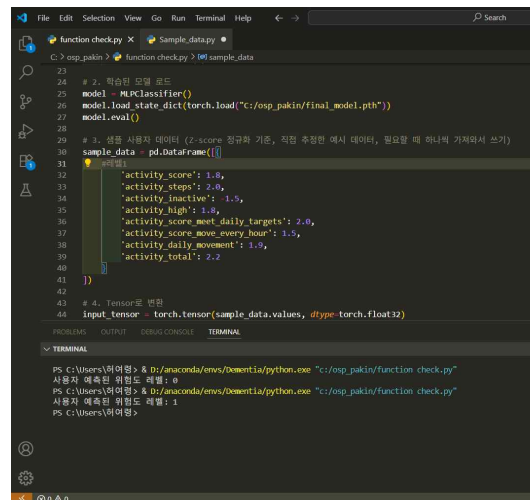
입력값은 실제 각 위험도 단계(레벨 0, 1, 2, 3)에 해당하는 임의의 샘플 데이터를 생성하고 모델에 입력하였다.

그 결과, 레벨 0~2까지는 정확하게 예측되지만 레벨 3을 2로 혼동하는 결과를 확인할 수 있었다. 결과는 동영상으로도 확인할 수 있다.



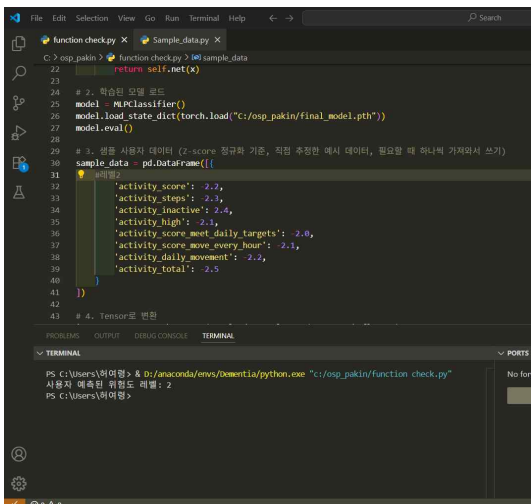
```
23 # 2. 학습된 모델 로드
24 model = MLPClassifier()
25 model.load_state_dict(torch.load("c:/osp_pakin/final_model.pth"))
26 model.eval()
27
28 # 3. 샘플 사용자 데이터 (Z-score 정규화 기준, 직접 주정한 예시 데이터, 필요할 때 하나씩 가져와서 쓰기)
29 sample_data = pd.DataFrame([
30     #레벨0
31     {'activity_score': 0.1,
32      'activity_steps': 0.2,
33      'activity_inactive': 0.3,
34      'activity_high': 0.4,
35      'activity_score_meet_daily_targets': -0.1,
36      'activity_score_move_every_hour': -0.2,
37      'activity_daily_movement': 0.0,
38      'activity_total': -0.1
39     })
40
41 # 4. Tensor로 변환
42 input_tensor = torch.tensor(sample_data.values, dtype=torch.float32)
43
44 PS C:\Users\허여정> & D:/anaconda/envs/Dementia/python.exe "c:/osp_pakin/function check.py"
사용자 예측된 위험도 레벨: 0
PS C:\Users\허여정>
```

[그림 9. 단계(레벨) 0]



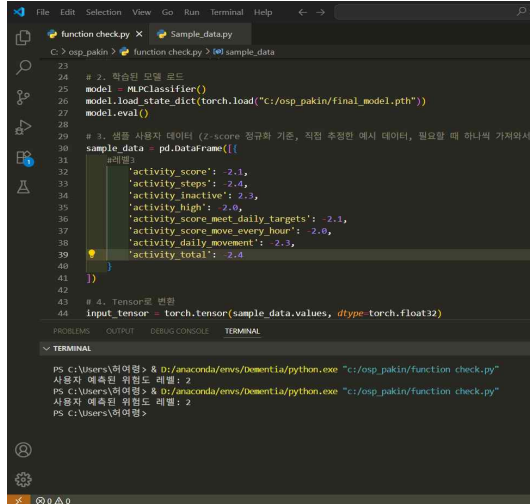
```
23 # 2. 학습된 모델 로드
24 model = MLPClassifier()
25 model.load_state_dict(torch.load("c:/osp_pakin/final_model.pth"))
26 model.eval()
27
28 # 3. 샘플 사용자 데이터 (Z-score 정규화 기준, 직접 주정한 예시 데이터, 필요할 때 하나씩 가져와서 쓰기)
29 sample_data = pd.DataFrame([
30     #레벨1
31     {'activity_score': 1.0,
32      'activity_steps': 2.0,
33      'activity_inactive': 1.5,
34      'activity_high': 1.8,
35      'activity_score_meet_daily_targets': 2.0,
36      'activity_score_move_every_hour': 1.5,
37      'activity_daily_movement': 1.0,
38      'activity_total': 2.2
39     })
40
41 # 4. Tensor로 변환
42 input_tensor = torch.tensor(sample_data.values, dtype=torch.float32)
43
44 PS C:\Users\허여정> & D:/anaconda/envs/Dementia/python.exe "c:/osp_pakin/function check.py"
사용자 예측된 위험도 레벨: 0
PS C:\Users\허여정> & D:/anaconda/envs/Dementia/python.exe "c:/osp_pakin/function check.py"
사용자 예측된 위험도 레벨: 1
PS C:\Users\허여정>
```

[그림 10. 단계(레벨) 1]



```
22 return self.net(x)
23
24 # 2. 학습된 모델 로드
25 model = MLPClassifier()
26 model.load_state_dict(torch.load("c:/osp_pakin/final_model.pth"))
27 model.eval()
28
29 # 3. 샘플 사용자 데이터 (Z-score 정규화 기준, 직접 주정한 예시 데이터, 필요할 때 하나씩 가져와서 쓰기)
30 sample_data = pd.DataFrame([
31     #레벨2
32     {'activity_score': 2.0,
33      'activity_steps': 2.3,
34      'activity_inactive': 2.4,
35      'activity_high': 2.1,
36      'activity_score_meet_daily_targets': 2.0,
37      'activity_score_move_every_hour': 2.1,
38      'activity_daily_movement': 2.2,
39      'activity_total': 2.5
40     })
41
42 # 4. Tensor로 변환
43 input_tensor = torch.tensor(sample_data.values, dtype=torch.float32)
44
45 PS C:\Users\허여정> & D:/anaconda/envs/Dementia/python.exe "c:/osp_pakin/function check.py"
사용자 예측된 위험도 레벨: 2
PS C:\Users\허여정>
```

[그림 11. 단계(레벨) 2]



```
23 # 2. 학습된 모델 로드
24 model = MLPClassifier()
25 model.load_state_dict(torch.load("c:/osp_pakin/final_model.pth"))
26 model.eval()
27
28 # 3. 샘플 사용자 데이터 (Z-score 정규화 기준, 직접 주정한 예시 데이터, 필요할 때 하나씩 가져와서 쓰기)
29 sample_data = pd.DataFrame([
30     #레벨3
31     {'activity_score': -2.1,
32      'activity_steps': -2.4,
33      'activity_inactive': 2.3,
34      'activity_high': 2.0,
35      'activity_score_meet_daily_targets': -2.1,
36      'activity_score_move_every_hour': -2.0,
37      'activity_daily_movement': -2.3,
38      'activity_total': -2.4
39     })
40
41 # 4. Tensor로 변환
42 input_tensor = torch.tensor(sample_data.values, dtype=torch.float32)
43
44 PS C:\Users\허여정> & D:/anaconda/envs/Dementia/python.exe "c:/osp_pakin/function check.py"
사용자 예측된 위험도 레벨: 2
PS C:\Users\허여정> & D:/anaconda/envs/Dementia/python.exe "c:/osp_pakin/function check.py"
사용자 예측된 위험도 레벨: 2
PS C:\Users\허여정>
```

[그림 12. 단계(레벨) 3]

- Github 사이트

- <https://github.com/heoryeong/Dementia-diagnosis/tree/main>

- 프로젝트 수행 결과의 한계점

- 개발 기술의 한계점

- 학습 데이터의 불균형으로 인해 class 0(치매 저위험군)의 예측은 초기 단계에서 거의 이루어지지 않았다. 해당 클래스의 비중이 매우 작아 모델이 유의미한 패턴을 학습하지 못했다. 이를 개선하기 위해 모델 구조 조정, 예폭·배치 크기 변경, SMOTE 적용, 클래스 가중치 보정 등의 방법을 시도하였다. 그 결과, 최종 모델에서는 class 0을 포함한 전체 클래스에 대해 보다 안정적인 예측 성능을 확보할 수 있었다. 다만 초기 데이터가 더 풍부했다면 일반화 성능은 더욱 높아졌을 것이다.
 - class 2(중상 위험군)과 class 3(고위험군) 사이에서 다소 혼동이 발생하는 경향이 있었다. 이는 두 클래스 모두 활동량이 낮고 비활동 시간이 길다는 공통적인 특성을 가지기 때문에, 정규화된 수치 상으로 경계가 불분명해지는 현상으로 해석된다. 특히 Z-score 기준에서는 극단적인 값일수록 구별이 어렵기 때문에, 이 구간에 속한 데이터들의 분포가 겹치는 현상이 원인 중 하나로 분석된다. 이를 해결하기 위해 추후 데이터 확보 및 클래스 간 특성 강화 학습을 통해 보완될 수 있을 것으로 기대된다.
 - 걸음걸이 변화는 치매의 여러 유형 중 일부에서만 유의미한 지표로 작용하기 때문에, 이를 기반으로 보다 일반적인 치매를 예측하는 데에는 본질적인 한계가 있었다. 이에 따르면 연구에서는 분석 범위를 파킨슨병 관련 치매로 한정하여 모델을 개발할 수밖에 없는 한계가 있었다.

- 프로젝트 수행의 어려움 및 느낀점

- 딥러닝 모델을 처음 구현해보는 과정에서 크고 작은 시행착오를 많이 겪었지만, 이를 해결하기 위해 다양한 딥러닝 모델링 기법을 조사하고, 학습 환경을 직접 설정해보는 경험을 통해 실질적인 기술 역량이 크게 향상되었다고 느꼈다. 단순히 이론에 머무르지 않고 실제로 적용하고 개선해나가는 과정을 겪음으로서 문제 해결 능력과 모델링에 대한 이해도가 함께 높아졌으며, 이러한 성장 과정을 통해 큰 성취감을 얻을 수 있었다.
 - 이번 프로젝트를 진행하면서 가장 어려웠던 점은 CSV 파일 형태의 데이터를 기반으로 딥러닝을 구성하고 학습시키는 작업이 처음이었다는 것이다. 특히 데이터 전처리 과정에서 어떤 변수를 선택해야 할지, 어떻게 정규화하고 라벨링할지를 결정하는 과정에서 많은 시행착오를 겪었다. 그러나 단순히 모델을 학습시키는 데 그치지 않고, 치매 위험도를 보다 객관적으로 분류하기 위해 SHAP 기반의 위험도 분석을 도입하고, 데이터에 근거한 정량적 판단 기준을 마련하고자 노력했다. 이러한 과정을 통해 딥러닝 모델이 단순한 예측 도구를 넘어, 설명 가능한 의료 보조 시스템으로 확장될 수 있음을 체감할 수 있었고, 데이터 기반 사고의 중요성을 깊이 느끼게 되었다.
 - 깃허브를 처음 사용해보았기 때문에, 팀원분이 자료를 올려주셔도 어디서 봐야 하는지 잘 몰랐고 파일 업로드 과정에서도 어려움을 겪었습니다. 그러나 팀 프로젝트를 진행하며 깃허브를 사용한 결과 점차 익숙해졌고 앞으로도 깃허브를 무리없이 활용할 수 있을 것 같습니다.
 - 이론으로만 간략하게 접했던 데이터 수집 및 라벨링 과정을 처음 직접 수행하면서 어려움을 겪었지만 팀원들을 도움 끝에 잘 완성할 수 있었다. 이번 프로젝트를 통해 협업의 중요성을 크게 느낄 수 있었고 오픈소스 활용에 대해 더 깊이 이해하게 되었다.